accessing a shared data structure, you *lock* the mutex associated with that data, and when you've finished accessing the data structure, you *unlock* the mutex. The Thread Library then ensures that once one thread has locked a specific mutex, all other threads that try to lock the same mutex have to wait until the thread that successfully locked the mutex unlocks it. This ensures that all threads see a self-consistent view of the shared data, without any broken invariants.

Mutexes are the most general of the data-protection mechanisms available in C++, but they're not a silver bullet; it's important to structure your code to protect the right data (see section 3.2.2) and avoid race conditions inherent in your interfaces (see section 3.2.3). Mutexes also come with their own problems, in the form of a *deadlock* (see section 3.2.4) and protecting either too much or too little data (see section 3.2.8). Let's start with the basics.

3.2.1. Using mutexes in C++

In C++, you create a mutex by constructing an instance of `std::mutex`, lock it with a call to the member function `lock()`, and unlock it with a call to the member function `unlock()`. However, it isn't recommended practice to call the member functions directly, because this means that you have to remember to call `unlock()` on every code path out of a function, including those due to exceptions. Instead, the Standard C++ Library provides the `std::lock_guard` class template, which implements that RAII idiom for a mutex; it locks the supplied mutex on construction and unlocks it on destruction, thus ensuring a locked mutex is always correctly unlocked. The following listing shows how to protect a list that can be accessed by multiple threads using a `std::mutex`, along with `std::lock_guard`. Both of these are declared in the `<mutex>` header.

Listing 3.1. Protecting a list with a mutex